

Service Overview

ภาพรวมของบริการ (Service Overview)

DisasterMonitoring Service

GitHub: [DisasterMonitoring_GitHub](https://github.com/tanatworks/CS366_MonitorDisasterService) / https://github.com/tanatworks/CS366_MonitorDisasterService

Video: [DisasterMonitoring_Video](#) /

<https://drive.google.com/file/d/13re71nDcUxifm7EhrhUnlPv3KuNjElBn/view?usp=sharing>

1. Service Owner

นายธนัช เกิดทิพย์ รหัสนักศึกษา 6609611980 ภาคปกติ

2. Service Purpose

Core Function: บริการรับผิดชอบการรวบรวม (Ingest) ข้อมูลปริมาณน้ำฝนและระดับน้ำจาก API/Sensor

ภายนอกแบบ Real-time นำมาประมวลผลเพื่อปรับสถานะภัยพิบัติ (NORMAL ⇌ WATCH ⇌ WARNING ⇌ CRITICAL)

ในแต่ละพื้นที่แบบอัตโนมัติ

Single Source of Truth: ทำหน้าที่เป็นแหล่งความจริงเดียวด้านสถานการณ์น้ำ เพื่อสนับสนุนการตัดสินใจของผู้
สั่งการ (Dispatcher)

Standalone Value: ระบบสามารถประเมินความเสี่ยงและอัปเดตสถานะภัยพิบัติได้อัตโนมัติตาม Threshold ที่
ตั้งไว้ แม้ระบบรับแจ้งเหตุส่วนกลางหรือระบบผู้สั่งการ (Dispatch) จะล่มก็ตาม

3. Pain Point ที่แก้ไข

- ปัญหาทั่วไป:

ข้อมูลระดับน้ำและสภาพอากาศกระจายอยู่ตามหลายแห่ง ไม่มีศูนย์รวมข้อมูล

- ปัญหาที่เกี่ยวข้องกับ Service:

Dispatcher ขาดข้อมูลแบบ Real-time ที่ถูกประมวลผลความเสี่ยงมาให้แล้ว ทำให้ต้องเสียเวลาเปิดดู

หลายหน้าจอเพื่อวิเคราะห์สถานการณ์เอง

- ผลกระทบหากไม่มี Service นี้:

การประเมินสถานการณ์จะล่าช้าและเกิด Human Error ได้ง่าย ส่งผลให้การออกประกาศเตือนภัยหรือการสั่งอพยพประชาชนทำได้ช้า ซึ่งอาจก่อให้เกิดความสูญเสียต่อชีวิตและทรัพย์สิน

3. Target Users

- Dispatcher (เจ้าหน้าที่):

ใช้สำหรับเฝ้าระวังและสั่งการปรับเปลี่ยนสถานะฉุกเฉิน (Manual Override)

- Citizen (ประชาชนในพื้นที่):

ใช้สำหรับดูข้อมูลระดับน้ำและสถานะแจ้งเตือนในพื้นที่ของตนเอง

- Frontend:

ระบบหน้าบ้านที่เรียก API (GET) ไปแสดงผลเป็น Dashboard

- Notification Service:

ระบบแจ้งเตือนที่รองรับ Async Event เพื่อนำไปส่ง SMS หรือ Push Notification

4. Service Boundary

- In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ):

- รับและจัดเก็บข้อมูลตัวเลขระดับน้ำและปริมาณฝน (Ingestion Data)
- ประมวลผลและจัดการสถานะภัยพิบัติ (Disaster Status State)
- ตรวจสอบความสดใหม่ของข้อมูล (Data Freshness / Outdated Flag)
- เปิดจ็อบชั่วคราว (Transactional Outbox) และยิงขอเปิดเหตุการณ์ไปยังระบบเพื่อน เมื่อสถานะเป็น CRITICAL
- รับ Webhook (SQS) จากระบบเพื่อน เพื่อนำ incident_id มาผูกกับพื้นที่ และรับแจ้งเมื่อเหตุการณ์ถูก CLOSED

- Out-of-scope / Not Responsible For (ไม่รับผิดชอบ)

- การส่งข้อความแจ้งเตือน SMS/Email ไปยังผู้ใช้ปลายทาง
- การจัดการข้อมูลพิกัดแผนที่แบบละเอียด หรือข้อมูลประชากรในพื้นที่
- การจัดการเส้นทางอพยพ หรือจัดการศูนย์พักพิง
- ไม่รับผิดชอบการสั่งการทีมกู้ภัย และไม่ทำหน้าที่รับแจ้งเหตุจากประชาชน

- **Independent Deployment:** ระบบถูกออกแบบให้สามารถเริ่มต้นการทำงาน (Start) และรับข้อมูล Ingestion Data ได้อย่างอิสระโดยไม่ต้องรอการตอบสนองจากบริการอื่น เพื่อป้องกันปัญหา Distributed Monolith

5. Autonomy / Decision Logic

บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ:

- **Auto-scaling Status:** ประเมินจาก water_level_cm และ rainfall_mm (เช่น ถ้าน้ำ > 150 cm ให้เป็น CRITICAL ทันที)
- **Data Freshness:** ประเมิน last_updated หากเซ็นเซอร์ไม่ส่งข้อมูลมานานกว่าเวลาที่กำหนด จะปรับสถานะเป็น Data Outdated
- **Dispatcher Priority:** หาก is_manual_override เป็น true ระบบจะระงับการคำนวณอัตโนมัติ และยึดสถานะที่เจ้าหน้าที่สั่งเป็นหลัก

การตัดสินใจอิงจาก:

- water_level_cm (ระดับน้ำ) และ rainfall_mm (ปริมาณน้ำฝน)
- last_updated (เวลาที่อัปเดตข้อมูลล่าสุด) เทียบกับเวลาปัจจุบัน
- is_manual_override (คำสั่งจาก Dispatcher ที่มี Priority สูงสุด)

บริการสามารถตัดสินใจได้เองภายใต้ business rules (เช่น ถ้าน้ำ > 150 cm ให้ปรับเป็น CRITICAL) โดยไม่ต้องรอการอนุมัติจากมนุษย์ในกรณีปกติ

6. Owned Data

- **Area Data (area_id, area_name, geo_location):**

ข้อมูลรหัส ชื่อ และพิกัดของพื้นที่ ซึ่งระบบได้รับมาจากเซ็นเซอร์ (Ingestion) และจัดเก็บไว้เป็น Primary Key เพื่อใช้อ้างอิงและแสดงผลจุดที่เกิดเหตุ

- **API Data (water_level_cm, rainfall_mm):**

ข้อมูลตัวเลขระดับน้ำและฝน อยู่ภายใต้บริการนี้เนื่องจากเป็นข้อมูลแกนหลักที่รับเข้าจากการ Ingest และเป็นปัจจัยตั้งต้น (Input) ในการประมวลผลความเสี่ยง

- **Disaster Status State (disaster_status, status_description):**

ข้อมูลสถานะการเตือนภัยเป็นผลลัพธ์จาก Decision Logic ของบริการนี้ (เช่น การปรับเป็น WARNING) จึงต้องอยู่ภายใต้บริการนี้เพื่อให้สามารถควบคุมความถูกต้องของการอัปเดตได้

- **Operational Metadata:**

(last_updated, is_outdated, is_manual_override, sync_status) ข้อมูลควบคุมการทำงาน เป็นสิ่งจำเป็นต่อการตรวจสอบย้อนหลัง การทำ Concurrency Control และการจัดการสถานะการส่งข้อมูลไปยัง ระบบส่วนกลาง (เช่น PENDING_RETRY, SUCCESS)

- **Database Architecture & Constraints:**

ใช้ DynamoDB ในรูปแบบ Private Schema (3 ตาราง ได้แก่ Disaster_Areas, Disaster_AuditLogs, Disaster_IncidentReports) ห้ามเปิดเผยหรือแชร์ Connection String ให้กับ Service อื่น โดยเด็ดขาด การแลกเปลี่ยนข้อมูลข้ามบริการต้องกระทำผ่าน REST API, Amazon SNS และ SQS Webhook เท่านั้น เพื่อป้องกันการเกิด Shared Database Anti-pattern

7. Linked Data (Reference Only)

ข้อมูลที่ระบบนำมาใช้อ้างอิงเพื่อเชื่อมโยงความสัมพันธ์ (Correlation) แต่ไม่ได้เป็นเจ้าของ:

- **incident_id:** รหัสเหตุการณ์ส่วนกลาง (Central UUID) อ้างอิงจาก Report Ingestion & Verification Service ซึ่งระบบจะได้รับรหัสนี้ผ่านทาง Asynchronous Webhook (SQS) นำมาจัดเก็บชั่วคราวเพื่อเชื่อมโยงความสัมพันธ์ ว่าพื้นที่ใดกำลังเกิดเหตุการณ์ภัยพิบัติหมายเลขใดอยู่ และเพื่อรับทราบเมื่อเหตุการณ์ถูกคลี่คลาย (CLOSED)
- อ้างอิงข้อมูล Incident_id จาก Central Incident Schema ของระบบส่วนกลาง โดยเก็บรหัสนี้ไว้เพื่อทำ Data Correlation และป้องกันการเกิด Data Duplication เมื่อมีการส่งข้อมูลสถานะซ้ำ

8. Non-Functional Requirements

- **API Standardization & Policies:**

- **API Versioning:** บังคับใช้ /v1/ ใน Base URL หรือ Path ของทุก API อย่างเคร่งครัด
- **Breaking Changes Policy:** ห้ามแก้ไขโครงสร้างข้อมูล ลบฟิลด์เดิม หรือเปลี่ยนประเภทข้อมูลใน API เวอร์ชันปัจจุบัน (v1) หากมีการเปลี่ยนแปลงที่ไม่เข้ากันย้อนหลัง (Breaking Changes) ต้องแจ้งเตือน Consumer ล่วงหน้าอย่างน้อย 30 วัน และให้สร้าง Endpoint ใหม่เป็น v2 แทน

- **Idempotency:**

Inbound: API รับข้อมูลขาเข้า (POST /ingest) ต้องตรวจสอบ timestamp เพื่อปฏิเสธการเขียนข้อมูลซ้ำ หรือการอัปเดตทับด้วยข้อมูลที่เก่ากว่า (Out-of-order messages)

- **Event Delivery & Asynchronous Processing:**

- **2 Queues Architecture:**

บริการรับ Webhook ผ่าน API Gateway ที่เชื่อมต่อกับ Amazon SQS โดยแยกคิวขาจากกัน (Events Queue และ Status Queue) พร้อมใช้ Worker 2 ตัวดึงข้อมูลแบบขนาน (Parallel Processing)

- **At-least-once Delivery & Long Polling:**

ใช้การดึงข้อมูลแบบ Long Polling (WaitTimeSeconds: 20) เพื่อลดภาระของระบบ และรับประกันการประมวลผลข้อมูลแจ้งเตือนสถานะอย่างน้อย 1 ครั้ง

- **Fault Tolerance (Fallback):**

- **Explicit Timeout:**

Explicit Timeout: บังคับตั้งค่าการรอคอยสูงสุดสำหรับการเรียก Synchronous Outbound Call ทุกเส้นทางที่ 30 วินาที

- **Synchronous Fallback (Outbox Pattern + Cron Job):**

หาก API ต้นทางล้มเกิน 15 นาที ระบบจะเปลี่ยนค่า is_outdated = true และในกรณีที่ระบบเพื่อน (Report Ingestion) ไม่ตอบสนองหรือล้ม ระบบเราจะไม่ Crash แต่จะเก็บคำสั่งลงฐานข้อมูล Disaster_IncidentReports ด้วยสถานะ PENDING_RETRY และปล่อยให้ Cron Job ทำการ Retry พร้อมกลไก Exponential Backoff อัตโนมัติ

- **Concurrency Control:**

การอัปเดตข้อมูลพร้อมกันต้องใช้กลไกป้องกัน Race Condition โดยเฉพาะการชนกันระหว่างระบบคำนวณอัตโนมัติ (Sensor) และการสั่ง Manual Override ของ Dispatcher

- **Performance:**

API แบบ Synchronous สำหรับดึงข้อมูลต้องรองรับปริมาณ Request ที่สูงมาก (Read-heavy) โดยออกแบบ Endpoint GET /api/v1/monitor-disaster/areas ให้เป็น Composite API ที่รวบรวมข้อมูลสถานะภัยพิบัติ, ระดับน้ำ, และเวลาอัปเดต ส่งกลับใน 1 Call เพื่อลดจำนวน Request จาก Frontend

- **Security:**

การเข้าถึง API สำหรับ Ingest ข้อมูลและปรับสถานะด้วยตนเอง รวมถึงการรับ Webhook จะถูกป้องกันด้วยระบบ Authentication โดยต้องระบุ X-API-Key เพื่อยืนยันตัวตนและสิทธิ์ก่อนดำเนินการเสมอ

- **Observability:**

- **Distributed Tracing:** บังคับส่งค่า X-Trace-Id ใน Header ของ HTTP Response ขากลับทุกเส้นทางการทำงาน
- **Structured Error Context:** บังคับเพิ่มโครงสร้าง "traceId": "<uuid>" ลงใน Error Response Payload ของ API
- **Optimized Logging:** Worker สำหรับ SQS จะถูกกำหนดให้พิมพ์ Log เฉพาะเมื่อมี Message เข้ามาเท่านั้น เพื่อลด Noise ในระบบและทำให้การติดตามร่องรอย (Traceability) ง่ายขึ้น

Sync Contract

Synchronous Function Contract

Base URL:

<https://p4j0q5vplh.execute-api.us-east-1.amazonaws.com>

API Contract #1: Ingest External Data

ข้อมูลทั่วไป

- **Name:** Ingest External Data
- **Method:** POST
- **Path:** /api/v1/monitor-disaster/ingest
- **Type:** Synchronous
- **Description:** ใช้รับข้อมูลระดับน้ำและฝนจากเซ็นเซอร์ (IoT) หรือ API ภายนอกแบบเรียลไทม์ เพื่อนำมาประมวลผลและปรับสถานะภัยพิบัติ (NORMAL, WATCH, WARNING, CRITICAL) อัตโนมัติ

Request Headers:

- Content-Type: application/json
- x-api-key: <sensor_secret_key>

Request Body:

```
{  
  
  "area_id": "TH-BKK-001",  
  
  "area_name": "Bangkok - Don Mueang",  
  
  "source_api": "IOT_SENSOR_V1",  
  
  "water_level_cm": 155.0,  
  
  "rainfall_mm": 120.0,  
  
  "timestamp": 1708560000000,  
  
  "geo_location": { "lat": 13.913, "lon": 100.596 }  
  
}
```

Validation:

- area_id, source_api, timestamp : Required
- water_level_cm, rainfall_mm : Number (≥ 0)

Response Headers:

- X-Trace-Id: <uuid>

Response

- Success Response (200 OK): กรณีบันทึกข้อมูลและอัปเดตสถานะสำเร็จ

```
{  
  
  "updated_status": "CRITICAL",  
  
  "sync_status": "PENDING_INCIDENT",  
  
  "traceId": "<uuid>"  
}
```

- Success: 200 OK (กรณีข้อมูลเก่า ไม่บันทึกซ้ำ - Out-of-order message)

```
{  
  
  "message": "Out-of-order data ignored",  
  
  "traceId": "<uuid>"  
}
```

- Error Response (401 Unauthorized): กรณีไม่ได้แนบ X-API-Key หรือรหัสผิด

```
{  
  
  "error": "UNAUTHORIZED",  
}
```

```
"traceId": "<uuid>"
```

```
}
```

- **Error Response (500 Internal Server Error):** กรณีมีข้อผิดพลาดระดับระบบ หรือส่งข้อมูลผิดประเภท

```
{
```

```
"error": "Cannot read properties of undefined (reading 'lat')",
```

```
"traceId": "<uuid>"
```

```
}
```

Dependency, Reliability และ Failure Handling

- **Dependency:** มีการเรียกใช้ External Service แบบ Synchronous (HTTP POST) ไปยัง **Report Ingestion & Verification Service** เฉพาะกรณีที่ประเมินสถานะภัยพิบัติแล้วพบว่าเข้าสู่ระดับ **CRITICAL** เพื่อขอเปิดตัวเหตุการณ์
- **Timeout:** กำหนดระยะเวลาการรอคอยสูงสุด (Explicit Timeout) ไว้ที่ **30 วินาที**
- **Failure Handling (Fallback):** หาก Endpoint ปลายทางล้ม, ไม่ตอบสนองภายในเวลาที่กำหนด, หรือตอบกลับด้วย HTTP 5xx ระบบจะ **ไม่ส่ง Error 500 กลับไปหาผู้เรียก (Sensor)** แต่จะตอบกลับเป็น 200 OK (พร้อมระบุ sync_status: "PENDING_RETRY") จากนั้นระบบจะใช้กลไก **Transactional Outbox Pattern** โดยบันทึกข้อมูลลงตาราง Disaster_IncidentReports ด้วยรหัสชั่วคราว (temp-) เพื่อให้ Background Cron Job ทำหน้าที่ Retry เบื้องหลัง
- **Retry Mechanism:** อาศัย Cron Job ที่ทำงานทุก 5 นาที ทำการส่งข้อมูลซ้ำด้วยอัลกอริทึม **Adaptive Exponential Backoff** (หน่วงเวลาทวีคูณ) จนกว่าจะสำเร็จ หรือสูงสุด 10 ครั้ง (Max Retries)
- **Idempotency:** รองรับการทำงานแบบ Idempotent โดยระบบจะสร้างและแนบค่า **X-IncidentTNX-Id** (Trace ID) ไปใน Request Header ทุกครั้ง เพื่อให้ระบบปลายทางใช้ตรวจสอบและป้องกันการเปิดเหตุการณ์ซ้ำซ้อน (Duplicate Protection) กรณีที่มีการ Retry เกิดขึ้น

API Contract #2: Get Area Status

ข้อมูลทั่วไป

- **Name:** Get Area Status
- **Method:** GET

- **Path:** /api/v1/monitor-disaster/areas/{area_id}
- **Type:** Synchronous
- **Description:** ดึงข้อมูลสถานการณ์น้ำและสถานะภัยพิบัติล่าสุดสำหรับ Frontend หรือ Citizen App

Request

Path/Query Param

- area_id (string, required) - รหัสอ้างอิงพื้นที่ที่ต้องการระบุ

Headers

- Accept: application/json

Body: ไม่มี

Response Headers:

- X-Trace-Id: <uuid>

Response

- **Success Response (200 OK):**

```
{  
  
  "area_id": "TH-BKK-001",  
  
  "area_name": "Bangkok - Don Mueang",  
  
  "disaster_status": "WARNING",  
  
  "incident_id": "019C774D-1AC5-758B-AE95-5CD4AEB89258",  
  
  "water_level_cm": 120.5,  
  
  "rainfall_mm": 50.0,  
  
  "is_outdated": false,  
  
  "last_updated": "2026-02-21T10:00:00.000Z",
```

```
"traceId": "<uuid>"

}
```

- **Error Response (404 Not Found):**

```
{

  "error": "Area not found",

  "traceId": "<uuid>"

}
```

Dependency / Reliability

- ไม่เรียก service อื่น
- Idempotent (เป็นการอ่านข้อมูล / Read-only)
- Timeout: 30s

API Contract #3: Override Disaster Status

ข้อมูลทั่วไป

- **Name:** Override Disaster Status
- **Method:** PATCH
- **Path:** /api/v1/monitor-disaster/areas/{area_id}/status
- **Type:** Synchronous
- **Description:** สำหรับ Dispatcher ปรับสถานะด้วยตนเอง (Manual Override) มีลำดับความสำคัญสูงกว่าระบบอัตโนมัติ (Sensor)

Request

Path/Query Param

- area_id (string, required) - รหัสอ้างอิงพื้นที่ที่ต้องการระบุ

Headers

- X-API-Key: <secret_key>
- Content-Type: application/json

Body:

```
{  
  
  "disaster_status": "CRITICAL",  
  
  "status_description": "พ่น้ำขึ้นน้ำพัง ต้องอพยพทันที",  
  
  "overridden_by": "dispatcher_01"  
}
```

Validation:

- disaster_status : Must be in [NORMAL, WATCH, WARNING, CRITICAL]
- status_description, overridden_by : Required

Response Header:

- X-Trace-Id: <uuid>

Response

- **Success Response (200 OK):**

```
{  
  
  "message": "Status overridden",  
  
  "sync_status": "PENDING_INCIDENT",  
  
  "traceId": "<uuid>"  
}
```

- **Error Response (404 Not Found):**

```
{
```

```
"error": "Area not found",
```

```
"traceId": "<uuid>"
```

```
}
```

- **Error Response (500 Internal Server Error):**

```
{
```

```
"error": "One or more parameter values were invalid...",
```

```
"traceId": "<uuid>"
```

```
}
```

Dependency / Reliability

- **Outbound Call:** หาก Dispatcher ปรับสถานะเป็น CRITICAL ระบบจะทำการเรียกใช้ POST /v1/reports ของ Report Ingestion & Verification Service เพื่อเปิดเหตุการณ์ฉุกเฉิน
- **Explicit Timeout:** บังคับตั้งค่าการรอคอยสูงสุด (Timeout) ที่ 30 วินาที
- **Reliability (Outbox Pattern):** ทำงานคู่กับ Outbox Pattern หากเฟ้นล้ม ระบบจะ Mark สถานะเป็น PENDING_RETRY เพื่อรอ Sync ข้อมูลไปหาเพื่อนในภายหลัง
- **Concurrency Control:** หากอัปเดตสำเร็จ จะปรับค่า is_manual_override = true เพื่อป้องกันไม่ให้ข้อมูลจากเซ็นเซอร์ (API #1) มาเขียนทับการตัดสินใจของเจ้าหน้าที่

API Contract #4: List All Areas

ข้อมูลทั่วไป

- **Name:** List All Areas
- **Method:** GET
- **Path:** /api/v1/monitor-disaster/areas
- **Type:** Synchronous

- **Description:** ใช้สำหรับดึงรายการพื้นที่ทั้งหมดในระบบพร้อมสถานะปัจจุบัน เพื่อให้ Frontend นำไปแสดงผลรวมบนหน้า Dashboard หรือหน้าแผนที่ และ เป็น **Composite API** ที่รวมข้อมูลจำเป็น (สถานะภัยพิบัติ, ระดับน้ำ, เวลาอัปเดต) ของทุกพื้นที่ ใน 1 Call เพื่อลดปัญหา **Chatty Services**

Request

Path/Query Param:

- Query Parameters: ไม่มี (None)

Headers:

- Accept: application/json

Body: ไม่มี

Response Headers:

- X-Trace-Id: <uuid>

Response

- **Success Response (200 OK):**

```
{  
  
  "areas": [  
  
    {  
  
      "area_id": "TH-BKK-001",  
  
      "disaster_status": "WARNING",  
  
      "water_level_cm": 120.5,  
  
      "is_outdated": false,  
  
      "last_updated": "2026-02-21T10:00:00.000Z"  
    }  
  ],  
}
```



```
"traceId": "<uuid>"

}
```

- **Error Response (500 Internal Server Error):**

```
{

  "error": "Internal server error message from DynamoDB",

  "traceId": "<uuid>"

}
```

Dependency / Reliability

- ไม่เรียก service อื่น
- Idempotent (Read-only)
- Timeout: 30s

API Contract #5: Get Area Historical Trends

ข้อมูลทั่วไป

- **Name:** Get Area Historical Trends
- **Method:** GET
- **Path:** /api/v1/monitor-disaster/areas/{area_id}/history
- **Type:** Synchronous
- **Description:** ใช้ดึงข้อมูลประวัติการเปลี่ยนแปลงระดับน้ำ ปริมาณฝน และสถานะภัยพิบัตีย้อนหลังของพื้นที่ที่ระบุ (คิวรีจากตาราง Disaster_AuditLogs) เพื่อให้บริการหน้าบ้าน (Frontend) นำไปวิเคราะห์แนวโน้ม (Trend Analysis) และแสดงผลเป็นกราฟ (Time-series Graph) บน Dashboard ให้เจ้าหน้าที่ประเมินสถานการณ์ล่วงหน้าได้

Request

Path/Query Param:

- area_id (string, required): รหัสพื้นที่ที่ต้องการดึงประวัติ

- hours (number, optional): จำนวนชั่วโมงย้อนหลังที่ต้องการดึงข้อมูล (ค่าเริ่มต้น: 24)

Headers:

- Accept: application/json

Body: ไม่มี

Response Headers:

- X-Trace-Id: <uuid>

Response

- Success Response (200 OK):

```
{  
  
  "area_id": "TH-BKK-001",  
  
  "timeframe_hours": 24,  
  
  "history": [  
  
    {  
  
      "timestamp": "2026-03-08T15:05:00Z",  
  
      "water_level_cm": 120.5,  
  
      "rainfall_mm": 80.0,  
  
      "disaster_status": "WARNING"  
    },  
  
    {  
  
      "timestamp": "2026-03-08T14:05:00Z",  
  
      "water_level_cm": 105.0,  
  
      "rainfall_mm": 50.0,
```

```
"disaster_status": "WATCH"

},

{

  "timestamp": "2026-03-08T13:05:00Z",

  "water_level_cm": 80.0,

  "rainfall_mm": 20.0,

  "disaster_status": "NORMAL"

}

]

}
```

- **Error Response (404 Not Found):**

```
{

  "error": {

    "code": "NOT_FOUND",

    "message": "Area ID not found or no history available",

    "traceId": "<uuid>"

  }

}
```

Dependency / Reliability

- ไม่เรียก service อื่น
- Idempotent (เป็นการอ่านข้อมูล / Read-only)
- คิวรีข้อมูลจาก DynamoDB (ตาราง Disaster_AuditLogs โดยใช้ area_id เป็น Index และเรียงลำดับตามเวลา)
- Timeout: 30s

Async Contract

Asynchronous Function Contract

Message Contract #1: Publish Disaster Incident Reported

ข้อมูลทั่วไป

- **Message Name:** DisasterStatusChanged
- **Interaction Style:** Publish-Subscribe (Fire-and-forget)
- **Producer:** DisasterMonitoring Service
- **Consumer:** Notification Service (ยังไม่มีคนทำ)
- **Channel/Topic:** MonitoringDisaster-Topic (Amazon SNS)
- **Version:** v1

คำอธิบาย

กระจายข่าว (Publish Event) อัปเดตสถานการณ์แบบ Asynchronous ทุกครั้งที่พื้นที่ใดพื้นที่หนึ่งมีการเปลี่ยนระดับความรุนแรง (NORMAL ⇌ WATCH ⇌ WARNING ⇌ CRITICAL)

ระบบจะทำงานแบบ Non-blocking โดยพ่น Event แจ้งเตือนออกไปที่ Amazon SNS ทันทีพร้อมแนบพิกัด (area_id) และระดับผลกระทบ (impact_level) เพื่อให้ระบบปลายทางนำข้อมูลไปประมวลผลต่อได้ทันที โดยระบบจะไม่หยุดรอเพื่อขอรับรหัส incident_id (กระบวนการรับรหัสเหตุการณ์จะถูกแยกไปจัดการใน Message Contract #2 ผ่าน SQS เพื่อป้องกันคอขวดและ Distributed Monolith)

Request

Message Headers (Metadata บน SNS/SQS)

- **messageType:** DisasterIncidentReported
- **eventId:** UUID (สร้างใหม่ทุกครั้งที่เกิด Event เพื่อให้ Message Broker ใช้ระบุตัวตนของข้อความและป้องกันการส่งซ้ำ - Deduplication)
- **publishedAt:** ISO-8601 datetime

Message Body

```
{  
  
  "event_id": "123e4567-e89b-12d3-a456-426614174000",  
  
  "timestamp": "2026-04-29T15:05:00.000Z",  
  
  "event_type": "DisasterStatusChanged",  
  
  "data": {
```

```
"area_id": "TH-BKK-001",

"old_status": "WARNING",

"new_status": "CRITICAL",

"incident_id": null,

"impact_level": 4

}

}
```

Field Definition:

Field	Type	Required	Description
event_id	UUID	Y	รหัสอ้างอิงการเปลี่ยนสถานะ (อ้างอิงจาก logId ในตาราง AuditLogs) ใช้เพื่อให้ Consumer ตรวจสอบและป้องกันการประมวลผลซ้ำ (Deduplication)
timestamp	UUID	Y	วันที่และเวลาที่เกิดการเปลี่ยนแปลง (ISO-8601)
event_type	String	Y	ประเภทของเหตุการณ์ (คือ "DisasterStatusChanged" เสมอ)
data	Object	Y	ก่อนข้อมูลหลักที่เกี่ยวข้องกับการเปลี่ยนสถานะ
data.area_id	String	Y	รหัสพื้นที่ที่เกิดเหตุ อ้างอิงตาม Location Service
data.old_status	String	Y	สถานะภัยพิบัติก่อนหน้า เพื่อให้ Consumer ใช้เช็ค State Transition
data.new_status	String	Y	สถานะภัยพิบัติใหม่ที่เพิ่งถูกปรับ (NORMAL, WATCH, WARNING, CRITICAL)
data.incident_id	UUID	N	รหัสอ้างอิงเหตุการณ์กลาง (Global ID) ที่ได้รับมาจาก Report Ingestion ผ่าน SQS (จะเป็น null หากระบบยังไม่ได้รับ Webhook กลับมา หรือยังไม่ถึงเกณฑ์ CRITICAL)

data.impact_level	Number	Y	ระดับความรุนแรงที่แปลงจาก Status (1 ถึง 4) เพื่อให้ระบบ Consumer อื่นนำไปใช้ประเมินผลได้ง่ายขึ้น
-------------------	--------	---	--

Validation Rules

- event_id ต้องเป็นรูปแบบ UUID ที่ไม่ซ้ำกัน
- data.area_id ต้องไม่เป็นค่าว่าง
- data.impact_level ต้องเป็นตัวเลข Integer (1 - 4)

Response Handling

- **Producer Behavior:** เนื่องจากเป็นรูปแบบ Broadcast Interaction (Fire-and-forget) ระบบ DisasterMonitoring จะรับผิดชอบแคโยนเข้า Topic สำเร็จ จะไม่รอรับ HTTP Response หรือ Payload ใดๆ กลับจาก Consumer
- **Consumer Behavior:** เมื่อบริการปลายทางได้รับ Event นี้ไป ระบบจะดึงข้อมูลไปประมวลผลต่ออย่างเป็นอิสระ (Decoupled)
- **Idempotency:** ฝั่ง Consumer ต้องนำ event_id ไปใช้เป็นตัวกุญแจตรวจสอบกับฐานข้อมูลของตนเอง เพื่อป้องกันการประมวลผลซ้ำซ้อน (Duplicate Processing) กรณีเกิดปัญหา Network Retry

Message Contract #2: Consume Incident Created Event

ข้อมูลทั่วไป

- **Message Name:** IncidentCreated
- **Interaction Style:** Message Queue (Long Polling / Asynchronous Pull)
- **Producer:** IncidentTracking Service (GCP)
- **Consumer:** DisasterMonitoring Service (AWS)
- **Channel/Queue:** monitor-disaster-events-queue (Amazon SQS)
- **Version:** v1

คำอธิบาย

รับข้อมูลตัวเหตุการณ์ที่ถูกเปิดสำเร็จจากระบบส่วนกลาง ทำหน้าที่นำรหัส incident_id (Central UUID) มาผูกเข้ากับ area_id และ report_id ในฐานข้อมูลท้องถิ่น (Disaster_Areas และ Disaster_IncidentReports) เพื่อให้ระบบใช้รหัสอ้างอิงเดียวกัน (Correlation ID) ในการอัปเดตสถานการณ์ครั้งต่อไป

Request

- **Message Headers:** Content-Type: application/json
- **Message Body:**

```
{  
  
  "eventType": "INCIDENT_CREATED",  
  
  "data": {  
  
    "source_report_id": "b46618db-234b-5544-8843-09748b99d525",  
  
    "incident_id": "inc-859c77de-6435-51d2-9ea9-d6529323c21a"  
  }  
}
```

Field Definition (Request Body)

Field	Type	Required	Description
eventType	String	Y	ประเภทของ Event ใช้เพื่อคัดกรอง Routing ภายใน Worker
data	Object	Y	ก้อนข้อมูลหลักของ Event
data.source_report_id	String	Y	รหัส Report ต้นทางที่ DisasterMonitoring เคยส่งไป (อ้างอิงจาดตาราง Disaster_IncidentReports)
data.incident_id	String	Y	รหัสเหตุการณ์ระดับส่วนกลาง (Global ID) ที่ถูกสร้างโดย IncidentTracking Service

Validation Rules

- eventType ต้องมีค่าเป็นตัวอักษร "INCIDENT_CREATED" อย่างถูกต้องแบบ Case-sensitive
- data.source_report_id และ data.incident_id ต้องไม่เป็นค่าว่าง (Not Null / Not Empty) หากขาดฟิลด์ใดฟิลด์หนึ่งระบบจะเข้าสู่กระบวนการ Reject ทันที
- ต้องมี source_report_id ปรากฏอยู่จริงในตาราง Disaster_IncidentReports ของฐานข้อมูล

Response Handling (Acknowledgement)

เนื่องจากการทำงานแบบ Asynchronous ข้ามคลาวด์ผ่าน Amazon SQS (Long Polling) ระบบจึงไม่มีการตอบกลับเป็น HTTP Response Body ไปยัง Producer โดยตรง แต่จะใช้กลไก Acknowledgement ตามมาตรฐาน Message Broker ดังนี้:

- **Success Case (Acknowledge):** เมื่อ Worker Task ดึงข้อความมาประมวลผลและอัปเดต incident_id ลงฐานข้อมูล (DynamoDB) สำเร็จ ระบบจะเรียก API DeleteMessage เพื่อลบข้อความออกจาก Queue ถาวร ถือเป็นการเสร็จสิ้น Transaction
- **Error/Reject Case (NACK & Retry):** หาก Payload ไม่ผ่าน Validation Rules หรือเกิด Exception ระหว่างการประมวลผล (เช่น หา source_report_id ในฐานข้อมูลไม่พบ) ระบบจะไม่ลบข้อความ และปล่อยให้ข้อความทริกเกอร์ Visibility Timeout เพื่อให้กลับเข้าคิวรอการ Retry หากล้มเหลวซ้ำจนเกินค่า Max Receive Count ข้อความจะถูกส่งไปยัง Dead Letter Queue (DLQ) อัตโนมัติเพื่อรอการตรวจสอบทางเทคนิค

Message Contract #3: Consume Incident Status Changed Event

ข้อมูลทั่วไป

- **Message Name:** IncidentStatusChanged
- **Interaction Style:** Message Queue (Long Polling / Asynchronous Pull)
- **Producer:** IncidentTracking Service (GCP)
- **Consumer:** DisasterMonitoring Service (AWS)
- **Channel/Queue:** monitor-disaster-status-queue (Amazon SQS)
- **Version:** v1

คำอธิบาย

รับข้อมูลการอัปเดตสถานะของเหตุการณ์จากระบบส่วนกลาง ทำหน้าที่ตรวจสอบสถานะล่าสุด หากเหตุการณ์ถูกประเมินว่าแก้ไขแล้ว (RESOLVED) หรือปิดเคสแล้ว (CLOSED) ระบบจะทำการเคลียร์ค่า incident_id (Set to Null) ออกจากตาราง Disaster_Areas เพื่อปลดล็อกพื้นที่ให้พร้อมรับการเปิดเหตุการณ์วิกฤตรอบใหม่ในอนาคต

Request

- Message Headers: Content-Type: application/json
- Message Body:

```
{  
  
  "eventType": "STATUS_CHANGED",  
  
  "data": {  
  
    "incident_id": "inc-859c77de-6435-51d2-9ea9-d6529323c21a",  
  
    "new_status": "CLOSED"  
  }  
}
```

Field Definition (Request Body)

Field	Type	Required	Description
eventType	String	Y	ประเภทของ Event ใช้เพื่อคัดกรอง Routing ภายใน Worker
data	Object	Y	ก้อนข้อมูลหลักของการอัปเดตสถานะ
data.incident_id	String	Y	รหัสเหตุการณ์ระดับส่วนกลาง (Global ID) ที่ถูกแก้ไขสถานะ
data.new_status	String	Y	สถานะล่าสุดของตัวเหตุการณ์ (เช่น IN_PROGRESS, RESOLVED, CLOSED)

Validation Rules

- eventType ต้องมีค่าเป็นตัวอักษร "STATUS_CHANGED" อย่างถูกต้องแบบ Case-sensitive
- data.incident_id และ data.new_status ต้องไม่เป็นค่าว่าง (Not Null / Not Empty)
- ต้องมี incident_id นี้ปรากฏอยู่จริงในตาราง Disaster_IncidentReports ของฐานข้อมูล หากไม่พบระบบจะทำการ Ignore (ข้ามการทำงานและลบข้อความ) โดยไม่เกิด Error

Response Handling (Acknowledgement)

เนื่องจากเป็นการทำงานแบบ Asynchronous ข้ามคลาวด์ผ่าน Amazon SQS (Long Polling) ระบบจึงไม่มีการตอบกลับเป็น HTTP Response Body ไปยัง Producer โดยตรง แต่จะใช้กลไก Acknowledgement ตามมาตรฐาน Message Broker ดังนี้:

- **Success Case (Acknowledge):** เมื่อ Worker Task ดึงข้อความมาประมวลผล (และทำการปิดตัวใน DynamoDB) สำเร็จ ระบบจะเรียก API DeleteMessage เพื่อลบข้อความออกจาก Queue ถาวร ถือเป็นการเสร็จสิ้น Transaction
- **Error/Reject Case (NACK & Retry):** หาก Payload ขาดฟิลด์ที่บังคับ หรือเกิด Exception ระดับ Database Connection ระบบจะไม่ลบข้อความ และปล่อยให้ข้อความทริกเกอร์ Visibility Timeout เพื่อให้กลับเข้าคิวรอการ Retry หากล้มเหลวซ้ำจนเกินค่า Max Receive Count ข้อความจะถูกส่งไปยัง Dead Letter Queue (DLQ) อัตโนมัติ

Service Data

Service Data

1) Area Disaster Operational State (ตาราง Disaster_Areas)

ข้อมูลตารางหลักที่เก็บสถานะล่าสุดของแต่ละพื้นที่ เพื่อใช้ในการตัดสินใจและแสดงผลแบบเรียลไทม์

Field Name	Type	Required	Description	Example
area_id	string	Y (Primary Key)	รหัสพื้นที่ (อ้างอิง Location Service)	TH-BKK-001
area_name	string	Y	ชื่อพื้นที่เพื่อความสะดวกในการแสดงผล	Bangkok - Don Mueang
disaster_status	enum	Y	สถานะภัยพิบัติ (NORMAL / WATCH / WARNING / CRITICAL)	WARNING
incident_id	uuid	N	รหัส Incident จากส่วนกลาง (รอรับจาก SQS Webhook)	550e8400-e29b-...33
water_level_cm	float	Y	ระดับน้ำล่าสุด ณ ปัจจุบัน	120.50
rainfall_mm	float	Y	ปริมาณน้ำฝนสะสมล่าสุด	185.20
geo_location	object	Y	พิกัดทางภูมิศาสตร์ {lat, lon}	{"lat": 13.91, "lon": 100.59}
status_description	string	N	รายละเอียด (ระบบใส่ให้ หรือ Dispatcher พิมพ์)	Updated via Sensor
is_outdated	boolean	Y	true ถ้าระบบไม่ได้รับ API อัปเดตเกิน 15 นาที	false
source_api	string	Y	รหัส API หรือแหล่งต้นทางที่ส่งข้อมูลมาให้	IOT_SENSOR_V1
is_manual_override	boolean	Y	true ถ้าระบุโดย Dispatcher (กันเซ็นเซอร์เขียนทับ)	false
last_timestamp	number	Y	Unix timestamp จาก	1708560000000

Field Name	Type	Required	Description	Example
			เซ็นเซอร์ ป้องกันข้อมูล Out-of-order	
last_updated	datetime	Y	เวลาที่ข้อมูลแถวนี้มีการแก้ไข ล่าสุดในระบบ	2026-02-21T10:00:00Z

2) Disaster Status Audit Log (ตาราง Disaster_AuditLogs)

ข้อมูลประวัติการเปลี่ยนแปลงสถานะ (History) เพื่อใช้ตรวจสอบย้อนหลัง และเป็นข้อมูลตั้งต้นสำหรับยิง Async Event แจ้งเตือนไปยัง SNS

Field Name	Type	Required	Description	Example
logId	uuid	Y (Primary Key)	รหัสบันทึกประวัติ (ใช้เป็น event_id ตอน Publish SNS)	550e8400-e29b-...
incident_id	uuid	N	รหัส Incident จาก ส่วนกลาง ณ เวลาที่เกิด Log (ถ้ามี)	550e8400-e29b-...33
area_id	string	Y	อ้างอิงรหัสพื้นที่ที่เกิดการเปลี่ยนแปลง	TH-BKK-001
previous_status	enum	Y	สถานะก่อนหน้า	WATCH
new_status	enum	Y	สถานะใหม่ที่ถูกเปลี่ยน	WARNING
water_level_cm	float	Y	ระดับน้ำ ณ เวลาที่บันทึกประวัตินี้	120.50
rainfall_mm	float	Y	ปริมาณน้ำฝน ณ เวลาที่บันทึกประวัตินี้	185.20
triggered_by	string	Y	รหัสผู้ส่งเปลี่ยน (Dispatcher) หรือ "SYSTEM_AUTO"	SYSTEM_AUTO
createdAt	datetime	Y	เวลาที่เกิดการเปลี่ยนแปลง	2026-02-21T10:00:00Z

Field Name	Type	Required	Description	Example
			สถานะ	

3) Incident Reports / Outbox (ตาราง Disaster_IncidentReports)

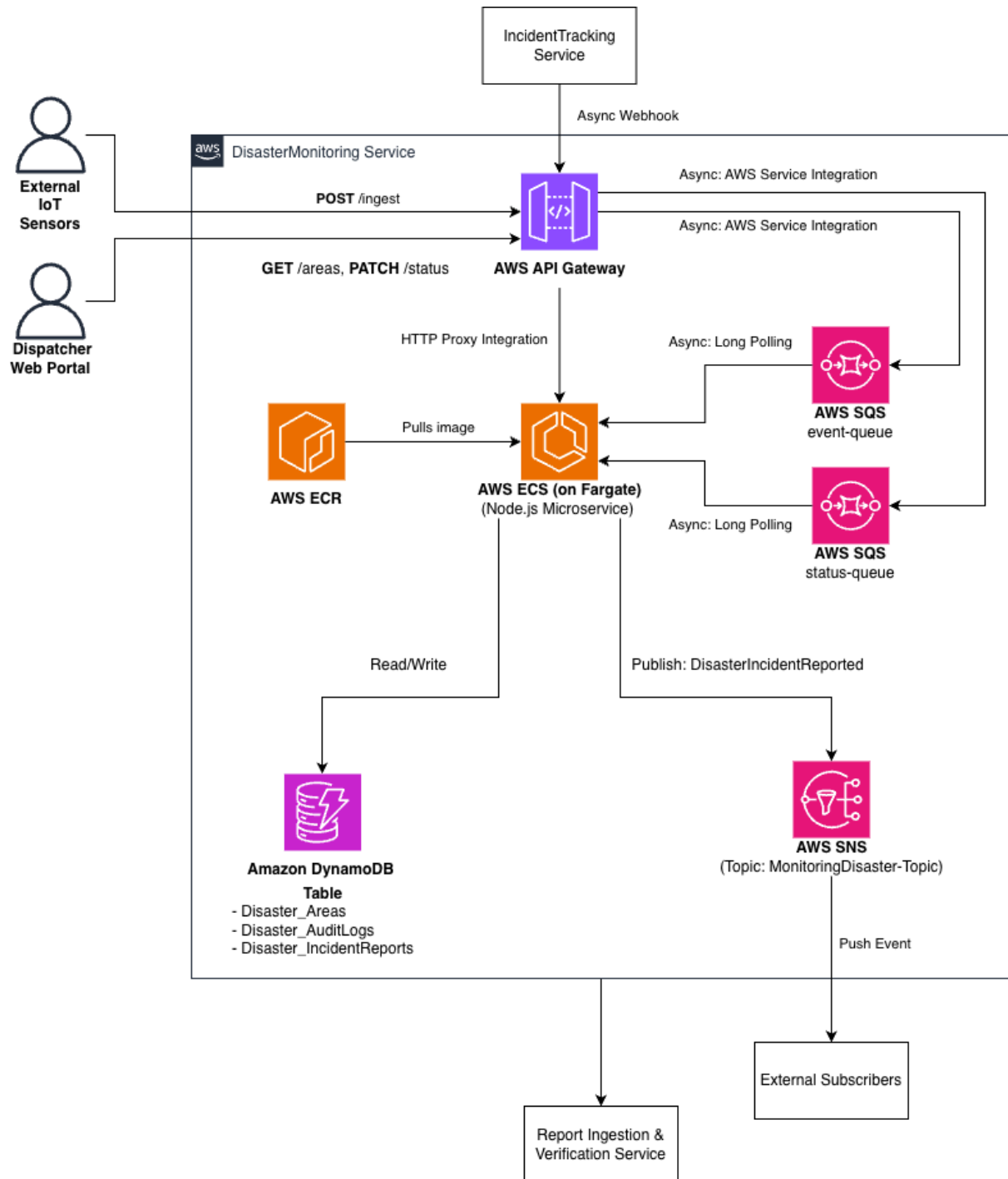
ตารางจัดการสถานะการยิง API ไปหาส่วนกลาง ทำหน้าที่เป็น Transactional Outbox รองรับระบบ Retry เมื่อ Network ชัดข้อง และรอจับคู่รหัสผ่าน SQS

Field Name	Type	Required	Description	Example
report_id	string	Y (Primary Key)	รหัสอ้างอิงของ Report (Local ID) เพื่อรอเปลี่ยนเป็นของจริง	temp-uuid หรือ r-12345
incident_id	uuid	N	รหัส Incident อ้างอิงระบบส่วนกลาง (Global ID) เติมผ่าน SQS	550e8400-e29b-...33
area_id	string	Y	อ้างอิงรหัสพื้นที่ที่เกิดการเปลี่ยนแปลง	TH-BKK-001
incident_type	string	Y	ประเภทของภัยพิบัติ (Service นี้กำหนดค่า "flood")	flood
incident_description	string	N	คำอธิบายเหตุการณ์ หรือ ข้อความจาก Dispatcher	[CRITICAL] พ่นก้นน้ำพัง...
impact_level	integer	Y	ระดับความรุนแรงตาม Schema กลาง (1-4)	4
threshold_water_cm	float	Y	ค่าระดับน้ำที่ทำให้ตัดสินใจเปิด Incident นี้	155.00
threshold_rain_mm	float	Y	ค่าปริมาณฝนที่ทำให้ตัดสินใจเปิด Incident นี้	120.50

Field Name	Type	Required	Description	Example
geo_location	object	Y	พิกัดทางภูมิศาสตร์ {lat, lon} สำหรับรายงาน	{"lat": 13.91, "lon": 100.59}
media_urls	list	N	รายการ URL รูปภาพ/วิดีโอ (ถ้ามี)	["http://image.jpg"]
report_source	string	Y	แหล่งข้อมูลที่ส่งเปิดเหตุการณ์ (IOT_SENSOR, OFFICIAL_APP)	IOT_SENSOR
reporter_id	string	Y	รหัสผู้รายงาน หรือชื่อเซ็นเซอร์	sensor-th-bkk-001
sync_status	enum	Y	สถานะเชื่อมต่อ (PENDING_RETRY, PENDING_INCIDENT, SUCCESS, CLOSED, FAILED_PERMANENTLY)	SUCCESS
idempotency_key	string	Y	Trace ID สำหรับตรวจสอบการยิงซ้ำ (Deduplication)	uuid-trace-1234
retry_count	integer	Y	จำนวนรอบที่ Retry ไปแล้ว (สำหรับ Cron Job)	0
next_retry_time	number	N	Unix timestamp สำหรับรอบการยิงซ้ำครั้งถัดไป (Backoff)	1708560300000
remote_trace_id	string	N	Trace ID ที่ตอบกลับมาจากบริการส่วนกลาง	remote-uuid-5678
reported_at	datetime	Y	เวลาที่ออกรายงาน Incident ฉบับนี้	2026-02-21T10:00:00Z
resolved_at	datetime	N	เวลาที่ได้รับสถานะ CLOSED จาก Webhook	2026-02-22T10:00:00Z

Service Architecture

Service Architecture



สถาปัตยกรรมนี้ถูกออกแบบมาเพื่อรองรับ NFR ด้าน High Availability และ Fault Tolerance โดยใช้ Amazon API Gateway ทำหน้าที่เป็น Proxy หน้าด่านเพื่อป้องกันระบบล่ม (Throttle/Rate Limit) และใช้ Amazon SQS ทำหน้าที่เป็น Buffer ดูดซับโหลดจาก Webhook ภายนอก ทำให้ Main API Task ใน ECS (Fargate) ไม่เกิดสภาวะ Resource Exhaustion

องค์ประกอบของระบบ (Infrastructure Components)

- Entry Point (Amazon API Gateway):

ทำหน้าที่เป็นหน้าด่านรับคำขอ (HTTP Request) โดยแบ่งการทำงานเป็น 2 ส่วนหลักคือ 1) ทำหน้าที่เป็น Reverse Proxy

(HTTP API) เพื่อส่งต่อคำขอจากผู้ใช้งานไปยัง ECS อย่างปลอดภัยพร้อมให้ URL ที่คงที่ถาวร และ 2) เป็นทางผ่านสำหรับรับ Webhook จากระบบภายนอกแล้วส่งตรงเข้าสู่คิว (SQS)

- **Compute (Amazon ECS on Fargate):**

ระบบประมวลผลหลักแบบ Serverless สำหรับรัน Application Node.js (ดึง Docker Image จาก Amazon ECR) ทำหน้าที่ 3 ส่วนพร้อมกัน ได้แก่ 1) ประมวลผล Business Logic หลักของ API 2) รัน Cron Job แบบ Exponential Backoff สำหรับทำ Retry และ 3) รัน Worker สองตัวแบบขนานเพื่อดึงข้อมูลจาก SQS (Long Polling)

- **Database (Amazon DynamoDB):**

ฐานข้อมูล NoSQL ที่มีความหน่วงต่ำ (Low Latency) เหมาะสำหรับการอ่านและเขียนแบบ Real-time แบ่งโครงสร้างเป็น 3 ตารางแบบ Private Schema ได้แก่ Disaster_Areas (สถานะพื้นที่), Disaster_AuditLogs (ประวัติ), และ Disaster_IncidentReports (Outbox)

- **Outbound Messaging (Amazon SNS):**

ทำหน้าที่เป็น Message Broker ขาออก สำหรับกระจายเหตุการณ์เตือนภัย (Publish Event) ออกไปยังระบบอื่น ๆ ในรูปแบบ Publish-Subscribe (Pub/Sub)

- **Inbound Messaging (Amazon SQS):**

คิวข้อความขาเข้า 2 ท่อ (events-queue และ status-queue) สำหรับรับ Asynchronous Webhook จากระบบส่วนกลาง ช่วยลดคอขวดและรองรับปริมาณทราฟฟิกพุ่งสูง (Spike) ได้อย่างมีประสิทธิภาพ

คำอธิบายการทำงาน (Workflow Explanation)

Flow 1: การประมวลผลข้อมูลและเชื่อมต่อส่วนกลาง (Synchronous Ingestion & Outbound API)

เมื่อเซ็นเซอร์ส่งข้อมูลระดับน้ำเข้ามาในระบบ คำขอจะถูกส่งผ่าน API Gateway ไปยัง ECS ระบบจะประเมินสถานการณ์ทันที (เช่น ถ้า น้ำ > 150 cm จะปรับเป็น CRITICAL) หากเป็นการเปลี่ยนแปลงระดับปกติ ระบบจะอัปเดตลง DynamoDB และตอบกลับ 200 OK แต่หากสถานะถึงขั้น CRITICAL ระบบจะใช้หลักการ Transactional Outbox Pattern โดยบันทึกคำสั่งขอเปิดเหตุการณ์ลงตาราง IncidentReports ก่อน จากนั้นจึงยิง HTTP POST ไปหา Report Ingestion Service ของระบบส่วนกลาง หากระบบปลายทางล้มหรือเกิด Timeout ระบบของเราจะไม่พังตาม แต่จะปล่อยให้ Cron Job ตื่นขึ้นมาทำการ Retry อัตโนมัติในภายหลัง (Fault Tolerance)

Flow 2: การกระจายข่าวสารเตือนภัย (Asynchronous Notification - SNS)

เมื่อระบบตรวจพบว่าสถานะของพื้นที่มีการเปลี่ยนแปลง (เช่น ปกติ ⇄ เฝ้าระวัง ⇄ วิกฤต) ระบบจะทำการสร้างประวัติการแก้ไข และส่ง Event ชื่อ DisasterStatusChanged ไปยัง Amazon SNS ในรูปแบบ Fire-and-forget ทันที โดยไม่รอการตอบกลับจากระบบอื่น วิธีนี้ทำให้ระบบ Notification สามารถ Subscribe และดึงข้อมูลไปดำเนินการต่อได้อย่างอิสระ (Decoupled Architecture)

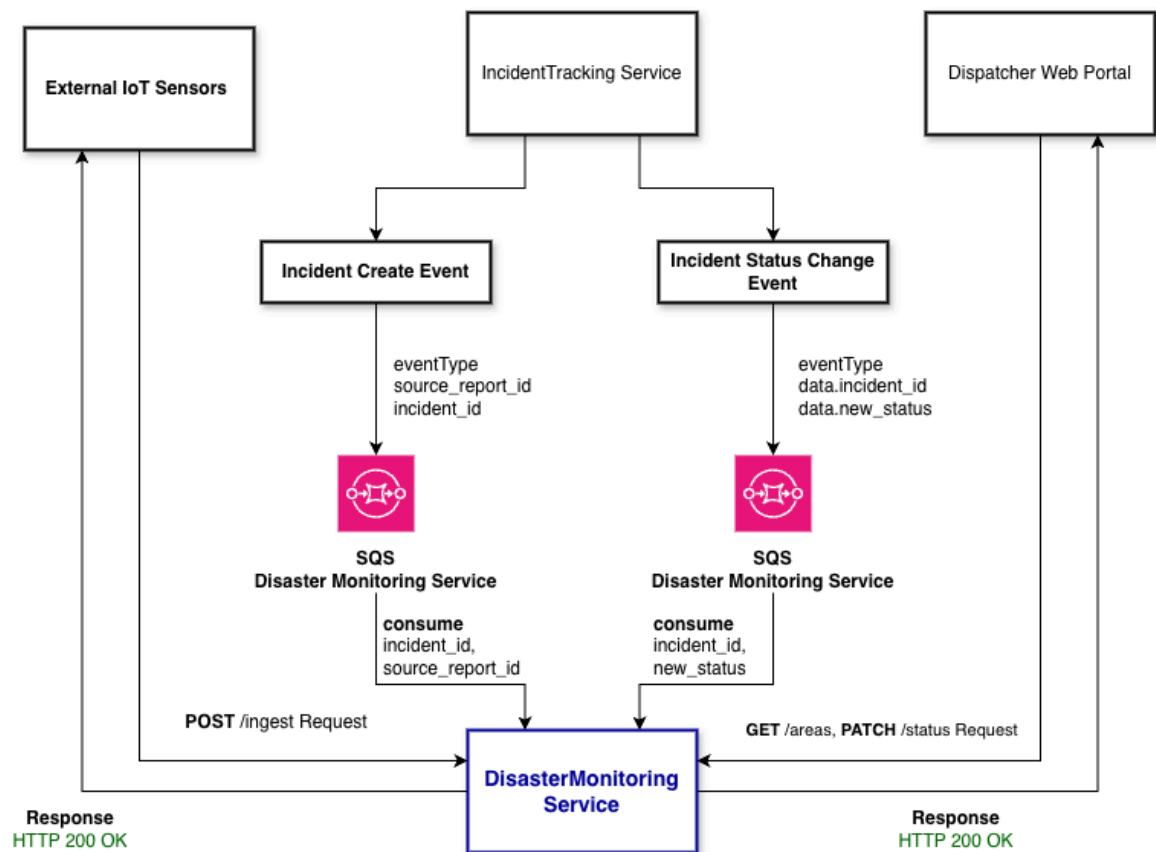
Flow 3: การประมวลผล Webhook ขาเข้า (Asynchronous Background Polling - SQS)

เมื่อระบบส่วนกลาง (Report Ingestion / Incident Tracking) สร้างตัวเหตุการณ์สำเร็จหรือปิดจ๊อบ จะมีการยิง Webhook ขากลับมาที่ระบบของเรา (ผ่าน API Gateway ทะลุเข้า Amazon SQS) ฝั่ง ECS จะมี Worker 2 ตัวที่ทำงานอยู่เบื้องหลังคอยดึงข้อมูลจาก SQS (Long Polling 20 วินาที) มาประมวลผล เพื่อนำรหัสเหตุการณ์ส่วนกลาง (incident_id) ไปผูกเข้ากับพื้นที่ในฐานข้อมูล หรือเคลียร์ค่าทิ้งเมื่อเหตุการณ์ถูกแก้ไขแล้วเสร็จ สถาปัตยกรรมนี้ทำให้ API ขาเข้าเส้นหลักไม่ต้องหยุดรอ (Non-blocking) และทำงานได้อย่างรวดเร็ว

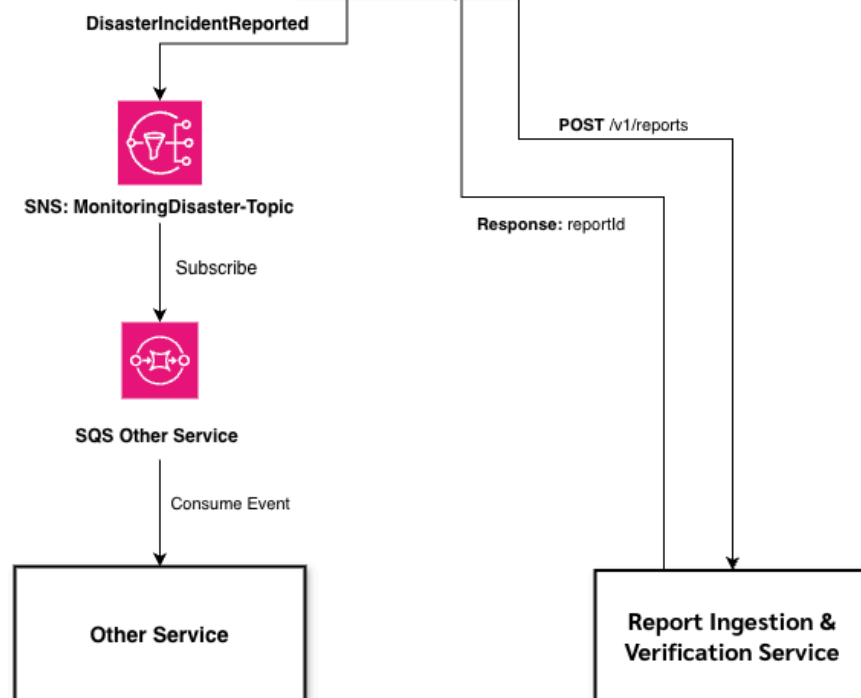
Service Interaction

Service Interaction

Up stream



Down stream



Upstream Services (บริการต้นทางที่เรียกใช้งาน Disaster Monitoring Service)

ระบบ DisasterMonitoring ทำหน้าที่เป็นระบบด้านหน้า (Edge Service) ในการรับข้อมูล (Data Ingestion) และเฝ้าระวังสถานการณ์น้ำท่วม โดยมีระบบต้นทางที่ส่งข้อมูลเข้ามามีดังนี้:

1. Data Sources (IoT Sensors / RID-API)

- **Integration:** Synchronous (HTTP Proxy Integration ไปยัง Main API Task)
- **Endpoint:** POST /api/v1/monitor-disaster/ingest
- **Behavior:** รับ Payload ข้อมูลระดับน้ำและปริมาณฝนแบบเรียลไทม์เพื่อประมวลผลและอัปเดตสถานะพื้นที่ลงฐานข้อมูลอย่างต่อเนื่อง

2. Dispatcher Web Portal

- **Integration:** Synchronous
- **Endpoint:**
 - GET /api/v1/monitor-disaster/areas
 - PATCH /api/v1/monitor-disaster/areas/{area_id}/status
- **Behavior:** ให้เจ้าหน้าที่ตั้งข้อมูลภาพรวม และส่งคำสั่ง Manual Override เพื่อบังคับเปลี่ยนสถานะภัยพิบัติ (มี Priority สูงกว่าการคำนวณอัตโนมัติของระบบ)

3. Incident Tracking Service

- **Integration:** Asynchronous (Webhook ผ่าน API Gateway สู่คิว Amazon SQS)
- **Mechanism:**
 - ระบบทำหน้าที่เป็น Consumer ใช้ Background Workers ดึงข้อมูล (Long Polling) จากคิว SQS ข้ามคลาวด์แพลตฟอร์มอย่างต่อเนื่อง
- **Data Correlation (Internal Processing):**
 - Worker Tasks ดึงข้อมูลมาประมวลผลเบื้องต้นตามเงื่อนไขดังนี้:
 - หากพบ Event INCIDENT_CREATED: สกัด source_report_id จาก Payload มาจับคู่กับ report_id เพื่ออัปเดต incident_id และปรับสถานะเป็น SUCCESS
 - หากพบ Event STATUS_CHANGED: เมื่อสถานะถูกระบุเป็น CLOSED จะทำการปิดวงจรเหตุการณ์ในฐานข้อมูลทันที

Downstream Services (บริการปลายทางที่ DisasterMonitoring Service ทำงานร่วมด้วย)

ระบบประเมินสถานการณ์และผลการทำงานกับบริการอื่นผ่าน Hybrid Interaction (Sync + Async) ดังนี้:

1. Report Ingestion & Verification Service (ระบบคัดกรองรายงาน)

- รูปแบบการสื่อสาร: Synchronous (HTTP POST)

- **Endpoint:** POST /v1/reports

- **Behavior:**

เมื่อระบบประเมินสถานการณ์น้ำอยู่ในเกณฑ์วิกฤต (CRITICAL) ระบบจะทำการเรียก API POST

/api/v1/reports ของบริการนี้ทันที เพื่อส่งข้อมูลดิบไปให้ประเมินความน่าเชื่อถือ

- **State Management:**

ระบบทำงานแบบ Non-blocking โดยรับเพียง report_id กลับมาบันทึกลง Amazon DynamoDB และตั้ง

สถานะในระบบเป็น PENDING_INCIDENT (ไม่มีการหยุดรอ incident_id เพื่อหลีกเลี่ยง Anti-Pattern แบบ Distributed

Monolith)

Dependency Mapping

Dependency Mapping – DisasterMonitoring Service

Service Dependencies (บริการที่ต้องพึ่งพา)

1. Report Ingestion & Verification Service

- **Type:** Service (Downstream)
- **Interaction Style:** Synchronous
- **Purpose:** ส่งข้อมูลระดับน้ำดิบไปตรวจสอบความน่าเชื่อถือทันทีเมื่อระบบประเมินสถานการณ์อยู่ในเกณฑ์วิกฤต (CRITICAL)
- **Criticality:** Non-Critical (สถาปัตยกรรมถูกออกแบบให้ทำงานแยกส่วนได้อิสระ)
- **Failure Handling:** หากบริการปลายทางล้ม ไม่ตอบสนอง หรือเกิด Explicit Timeout (เช่น เกิน 30 วินาที) ระบบจะใช้กลไก Local Outbox Pattern บันทึกข้อมูลลงฐานข้อมูลตนเองโดยกำหนดสถานะเป็น PENDING_RETRY ระบบจะไม่ถูก Block การทำงาน และดำเนินการรับข้อมูล Ingestion ต่อไปได้โดยไม่ต้องรอรหัส incident_id

2. Incident Tracking Service

- **Type:** Service (Upstream)
- **Interaction Style:** Asynchronous Webhook (via Amazon SQS)
- **Purpose:** พึ่งพาเพื่อรอรับค่า Global **incident_id** กลับมาผูกกับพื้นที่ และรอรับสถานะเมื่อเหตุการณ์ถูกปิด (CLOSED)
- **Criticality:** Non-Critical
- **Failure Handling:** หากบริการนี้ล้มหรือไม่ส่งข้อมูลมา ระบบย่อยของเราจะยังคงทำงานรับค่าเซ็นเซอร์ (Ingestion) และเปลี่ยนสถานะพื้นที่ตามตรรกะของตนเองต่อไปได้อย่างอิสระ (Standalone)

Infrastructure Dependencies (ระบบโครงสร้างพื้นฐานที่ต้องพึ่งพา)

3. Amazon SQS (events-queue และ status-queue)

- **Type:** Queue
- **Interaction Style:** Asynchronous (AWS Direct Integration & Long Polling)
- **Purpose:** ทำหน้าที่เป็นบัฟเฟอร์รองรับโหลด Webhook ข้ามคลาวด์จาก Incident Tracking Service (รับ Event INCIDENT_CREATED และ STATUS_CHANGED) เพื่อให้ระบบหลังบ้าน (Worker Tasks) ดึงไปอัปเดตข้อมูลแบบ Asynchronous
- **Criticality:** Critical (สำหรับรักษา Data Correlation แบบข้ามระบบ)

- **Failure Handling:** หาก Worker Task ประมวลผลล้มเหลว (เช่น จับคู่ report_id ไม่พบในฐานข้อมูล) ข้อความจะทริกเกอร์ Visibility Timeout และให้ระบบทำการ Retry อัตโนมัติ หากล้มเหลวเกินโควต้า (Max Receive Count) ข้อความจะถูกย้ายไปยัง Dead Letter Queue (DLQ) ทันทีเพื่อป้องกัน Poison Pill Message และรอการแก้ไขโดยระบบหรือผู้ดูแล

4. Amazon SNS (MonitoringDisaster-Topic)

- **Type:** Pub/Sub Topic
- **Interaction Style:** Asynchronous (Fire-and-forget)
- **Purpose:** กระจาย Event การเปลี่ยนสถานะพื้นที่ให้ระบบอื่นๆ นำไปใช้งาน
- **Criticality:** Moderate
- **Failure Handling:** หากระบบไม่สามารถ Publish เข้า SNS ได้เนื่องจากปัญหา Network หรือสิทธิ์ IAM ระบบจะบันทึก Log ข้อผิดพลาดไว้และทำงานส่วนอื่นต่อไปโดยไม่ขัดจังหวะ Transaction หลักของฝั่ง Ingestion

5. Amazon DynamoDB

- **Type:** Database
- **Interaction Style:** Internal Data Access (Read/Write)
- **Purpose:** ทำหน้าที่เป็น Single Source of Truth จัดเก็บข้อมูลสถานะความรุนแรงของพื้นที่ (Areas Table), ประวัติการเปลี่ยนแปลงสถานะ (AuditLogs Table), และประวัติการบันทึกเหตุการณ์ (IncidentReports Table)
- **Criticality:** Critical (ระบบประมวลผลหยุดทำงานทันทีหาก Database เข้าถึงไม่ได้)
- **Failure Handling:** หากเกิด Write Failure หรือเผชิญกับเงื่อนไข Provisioned Throughput Exceeded (Throttling) ระบบจะใช้กลไก Exponential Backoff ผ่าน AWS SDK เพื่อหน่วงเวลาและพยายามเขียนซ้ำ หากล้มเหลวถึงขีดสุด จะยกเลิกกระบวนการและโยน Exception (HTTP 500) กลับไปยัง Upstream Initiator ทันที